

Frontend Developer Guide: Biometric Face-recognition

Project Name: FARO (Face-base Access and Room Observer)

Framework: React + Next.js + Websocket

Introduction & Overview

1.1 โปรเจกต์คืออะไร? (What is this?)

Face-base Access and Room Observer คือ Web Application สำหรับระบบยืนยันตัวตนด้วยใบหน้า (Biometric Face Authentication) และระบบตรวจจับ/ระบุตัวบุคคลแบบ Real-time ภายในห้องเรียนหรือพื้นที่ควบคุม

ระบบทำงานโดย:

- เปิดกล้องผ่าน Browser (WebRTC)
- ส่งภาพแบบ real-time ผ่าน WebSocket
- Backend ประมวลผลด้วย AI (Face Detection + Face Recognition)
- ส่งผลลัพธ์กลับมาแสดงบนหน้าเว็บทันที

1.2 วัตถุประสงค์ของระบบ (System Objectives)

- ยืนยันตัวตนผู้ใช้งานด้วยใบหน้า
- ตรวจจับและระบุตัวบุคคลแบบ Real-time
- แสดงผลจำนวนคนเข้า-ออกห้อง
- รองรับการใช้งานผ่าน Browser ทั้ง Desktop และ Mobile

1.3 Frontend มีหน้าที่อะไร?

1. รัน Web-App Authentication ที่ประกอบไปด้วยหน้า Register, Login, Profile, verify, Identify
2. เปิดกล้องผู้ใช้งาน (navigator.mediaDevices.getUserMedia)
3. แสดงภาพกล้องแบบ Live Stream
4. จับภาพทุกช่วงเวลา (เช่น 100-300ms)
5. แปลงภาพเป็น JPEG Blob
6. ส่งภาพผ่าน WebSocket ไปยัง Backend
7. รับผลลัพธ์ JSON (ชื่อ, similarity, bounding box ฯลฯ)

8. แสดงผลบนหน้าจอแบบ Real-time สำหรับ Admin

2. Frontend Architecture

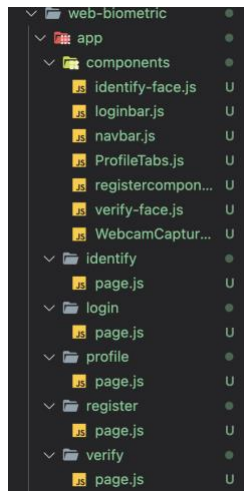
Frontend พัฒนาด้วย:

- Next.js (v16.1.6)
ใช้สำหรับสร้าง React-based Web Application และจัดการ routing, build system และ optimization
- React (v19.2.3)
ใช้สำหรับพัฒนา UI แบบ Component-based และจัดการ state ของระบบ
- react-webcam (ตรวจสอบ version จาก package.json)
ใช้สำหรับเข้าถึงกล้องผ่าน WebRTC abstraction layer
- Node.js (v22.14.0)
ใช้เป็น JavaScript runtime สำหรับพัฒนาและรัน Next.js application (development & build process)
- WebSocket API (Browser Native)
ใช้สำหรับสื่อสารแบบ real-time กับ Backend
- Canvas API
ใช้สำหรับจับภาพ (frame capture) และปรับขนาดก่อนส่งไป Backend
- WebRTC (getUserMedia)
ใช้สำหรับเข้าถึงกล้องจาก Browser

หมายเหตุ : คนต่อไปใช้ Node 18 อาจเกิด error ได้ ควรปรับใช้ตามที่ยกตัวอย่างหรือปรับแต่งได้ตามความเหมาะสม

2.1 Component Structure (ตัวอย่างโครงสร้าง)

Frontend ของ FARO ใช้โครงสร้างแบบ App Router ของ Next.js และพัฒนา UI ด้วย React โครงสร้างหลักแนะนำให้จัดแบบแยกตามหน้าที่ (Separation of Concerns)



(รูปภาพข้อ 2.1)

/app/identify/page.js

หน้าที่:

- เป็นหน้า Identify หลัก
- เรียกใช้ WebcamCapture
- เรียก WebSocket เพื่อส่งภาพ
- แสดงผลการระบุตัวบุคคล

เป็น orchestration layer ไม่ควรใส่ logic ประมวลผลภาพหนัก ๆ ในไฟล์นี้

/app/login/page.js

หน้าที่:

- ฟอर्म Login
- ส่ง username/password ไป backend
- รับ JWT token
- จัดการ redirect หลัง login สำเร็จ

/app/verify/page.js

หน้าที่:

- ใช้สำหรับ verify ใบหน้าหลังสมัคร

- เรียก verify-face component
- แสดงผลสำเร็จ/ล้มเหลว

/app/profile/page.js

หน้าที่:

- แสดงข้อมูลผู้ใช้
- แสดง Face status
- ใช้ ProfileTabs component

****Components Folder****

WebcamCapture.js

หน้าที่:

- เปิดกล้องด้วย getUserMedia
- แสดง video stream
- capture frame
- แปลงเป็น blob ก่อนส่ง

ไฟล์นี้เป็น core ของระบบ biometric

identify-face.js

หน้าที่:

- จัดการ logic การส่งภาพไป backend
- รับผลลัพธ์ recognition
- update state เพื่อแสดงผล

verify-face.js

หน้าที่:

- ใช้สำหรับ verify identity
- ส่งภาพใบหน้าไป backend
- ตรวจสอบว่า match กับ user หรือไม่

registercomponent.js

หน้าที่:

- รับข้อมูลสมัครสมาชิก
- จัดการ UI ของ register form

ProfileTabs.js

หน้าที่:

- จัด layout tab ในหน้า profile
- แยกข้อมูลเป็นหมวดหมู่

loginbar.js

หน้าที่:

- UI ส่วน login shortcut
- แสดงสถานะผู้ใช้

2.2 Identify Page Lifecycle

เมื่อผู้ใช้เข้าสู่หน้า /Identify ระบบจะทำงานตามลำดับดังนี้:

1. Component mount
2. ขอ permission กล้อง
3. เปิด WebSocket connection
4. เริ่ม capture frame ทุก 200–300ms
5. ส่งภาพไป Backend
6. รับผลลัพธ์ JSON
7. แสดง bounding box + name

8. เมื่อออกจากหน้า → stop camera + close socket

นี่คือหัวใจของระบบ ต้องเข้าใจ flow นี้ก่อนแก้โค้ด

Step 1: Component Mount

- useEffect ถูกเรียก

Step 2: Request Camera Permission

- navigator.mediaDevices.getUserMedia()

Step 3: Connect WebSocket

- new WebSocket("wss://domain/ws/identify")

Step 4: Start Frame Capture

- ใช้ setInterval ทุก 200–300ms
- drawImage ลง Canvas
- แปลงเป็น JPEG blob

Step 5: Send to Backend

- ws.send(blob)

Step 6: Receive JSON Result

```
{
  "track_id": 1,
  "name": "Student A",
  "similarity": 0.87,
  "bbox": [x1, y1, x2, y2]
}
```

Step 7: Render Overlay

- วาด bounding box
- แสดงชื่อ
- แสดง similarity

Step 8: Component Unmount

- stop camera stream
- clear interval
- close WebSocket

2.3. WebSocket Contract

- Frontend คาดหวัง JSON รูปแบบ:

```
{
  "track_id": number,
  "name": string,
  "similarity": number,
  "bbox": [number, number, number, number]
}
```

- กรณีไม่พบใบหน้า:

```
{
  "name": null
}
```

Frontend ต้อง validate response ก่อน render

3. State Management Strategy

ใช้ React Hooks:

useRef

- video element
- canvas element
- WebSocket instance

useState

- recognizedName
- similarity
- connectionStatus

หลีกเลี่ยง:

เก็บ frame ใน state

re-render ทุกครั้งที่ capture

4. Performance Guidelines

ค่าที่แนะนำ:

Parameter	Recommended
Resolution	640x480
FPS	3-5
Interval	200-300ms
JPEG Quality	0.6-0.8

5. Error Handling Strategy

Camera Permission Denied

- แสดงข้อความ error
- ไม่เริ่ม WebSocket

WebSocket Error

- แสดง connection error
- ทำ reconnect (optional)

Backend Timeout

- แสดง loading state
- ป้องกัน UI ค้าง

6.Security Considerations

Frontend ต้อง:

- ไม่เก็บ secret key
- ไม่ hardcode token
- ใช้ HTTPS
- Validate response format

หมายเหตุ:

Token แก้ใน DevTools ได้ แต่ไม่มีผลเพราะ Backend ตรวจสอบ signature

7.Environment Setup

Requirements

- Node.js >= 22
- npm >= 10

Install

- npm install
- npm run dev

คำแนะนำ:เมื่อเราจะ Dev ควรแยกออกจากDocker แล้วใช้ npm run dev เพื่ออัปเดตหน้าเว็บแบบเรียลไทม์ โดยไม่ต้องปิดเปิด Docker ใหม่ เมื่อทุกอย่างเสร็จสมบูรณ์แบบค่อยอัปเดตDocker ให้รันแทน

8. Known Limitations

- ระบบขึ้นกับคุณภาพแสง
- ถ้า resolution สูง CPU จะเพิ่ม
- WebSocket ต้อง stable network
- ไม่รองรับ multi-camera ใน version นี้

9. Future Improvements

- Multi-camera support
- WebWorker สำหรับ encode ภาพ
- Dynamic FPS adjustment
- Face enrollment preview
- Analytics dashboard

10. Critical Notes for Next Developer

ระบบนี้เป็น Real-time AI Streaming Application

สิ่งที่สำคัญที่สุด:

- Latency
- Frame size
- WebSocket stability

- Render optimization
- ถ้าแก้ logic โดยไม่เข้าใจ 4 ข้อนี้
ระบบจะช้าและ CPU พุ่งทันที

11. การติดตั้งและเริ่มใช้งาน (Installation & Setup)

การติดตั้งและเริ่มใช้งาน (สำหรับนักพัฒนาที่จะ Clone โปรเจกต์ต่อ)

เอกสารส่วนนี้อธิบายขั้นตอนการเตรียมสภาพแวดล้อม (Environment Setup) และวิธีรันโปรเจกต์ FARO สำหรับนักพัฒนาคนใหม่

11.1 Prerequisites (สิ่งที่ต้องมี)

ก่อนเริ่มต้น โปรดตรวจสอบว่าเครื่องของคุณติดตั้งซอฟต์แวร์ต่อไปนี้เรียบร้อยแล้ว:

1. Node.js

- แนะนำเวอร์ชัน: **v22.0.0 ขึ้นไป**
- ตรวจสอบเวอร์ชัน:

2. npm / yarn / pnpm

- npm จะติดตั้งมาพร้อม Node.js

3. Git

- ใช้สำหรับ Clone โปรเจกต์จาก Repository

11.2 Clone Repository

1. `git clone <repository_url>`
2. `cd web-biometric`

11.3 ติดตั้ง Dependencies

หลังจากเข้าโฟลเดอร์โปรเจกต์แล้ว ให้ติดตั้งแพ็คเกจทั้งหมด:

- npm install

หรือถ้าใช้ yarn:

- yarn install

11.4 การรัน Development Server

1. npm run dev

เมื่อรันสำเร็จ Terminal จะแสดง URL เช่น:

2. http://localhost:3000

ให้เปิด Browser ไปที่ URL ดังกล่าว

คำแนะนำ: หากรันฝั่ง frontend สำเร็จแล้วให้รันฝั่ง Backend ควบคู่ไปด้วยโดยใช้คำสั่ง docker compose up --build

11.5 การ Build สำหรับ Production

เมื่อพัฒนาเสร็จแล้ว:

- npm run build

จากนั้นสามารถรัน:

- npm start

หรือ Deploy ไปยัง:

- Vercel
- Docker
- Cloud Server

11.6 Troubleshooting (ปัญหาที่พบบ่อย)

กล้องไม่ทำงาน

- ตรวจสอบว่า Browser อนุญาต Camera
- ต้องเปิดผ่าน HTTPS หรือ localhost

WebSocket เชื่อมต่อไม่ได้

- ตรวจสอบว่า Backend รันอยู่

CORS Error

- ตรวจสอบว่า Backend เปิด CORS สำหรับ Frontend domain แล้ว

12. ตัวอย่างการใช้งาน

12.1 หน้า Register หรือ Enrollment

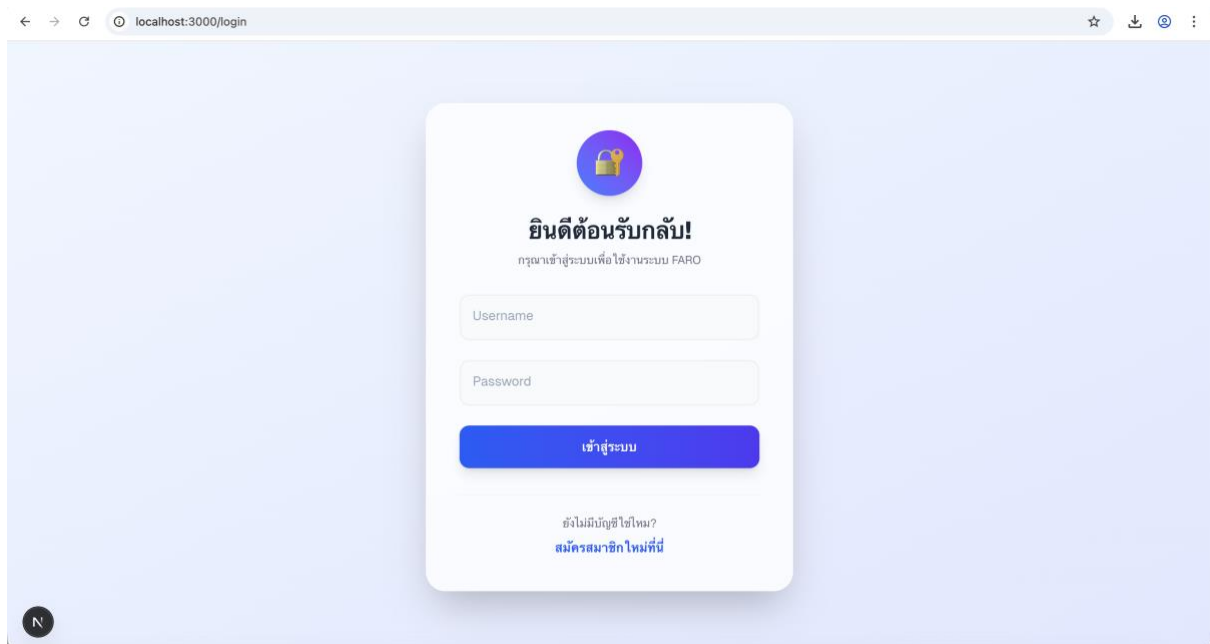
The screenshot shows a web browser at localhost:3000/register. The page features a 'Face Enrollment' section on the left with a green box indicating 'บันทึกครบแล้ว!' (All information recorded!) and a link to 'ย้ายไปยังหน้าถัดไป' (Move to next page). The main section is titled 'สร้างบัญชีผู้ใช้' (Create User Account) and contains the following fields:

- Username: test
- Password: test
- Confirm Password: test
- Email: test@gmail.com
- Phone Number: 0123456789
- Date of Birth: 01/01/2004

Below the fields is a blue button labeled 'ยืนยันการสมัครสมาชิก' (Confirm Registration) and a link labeled 'เข้าสู่ระบบ' (Login).

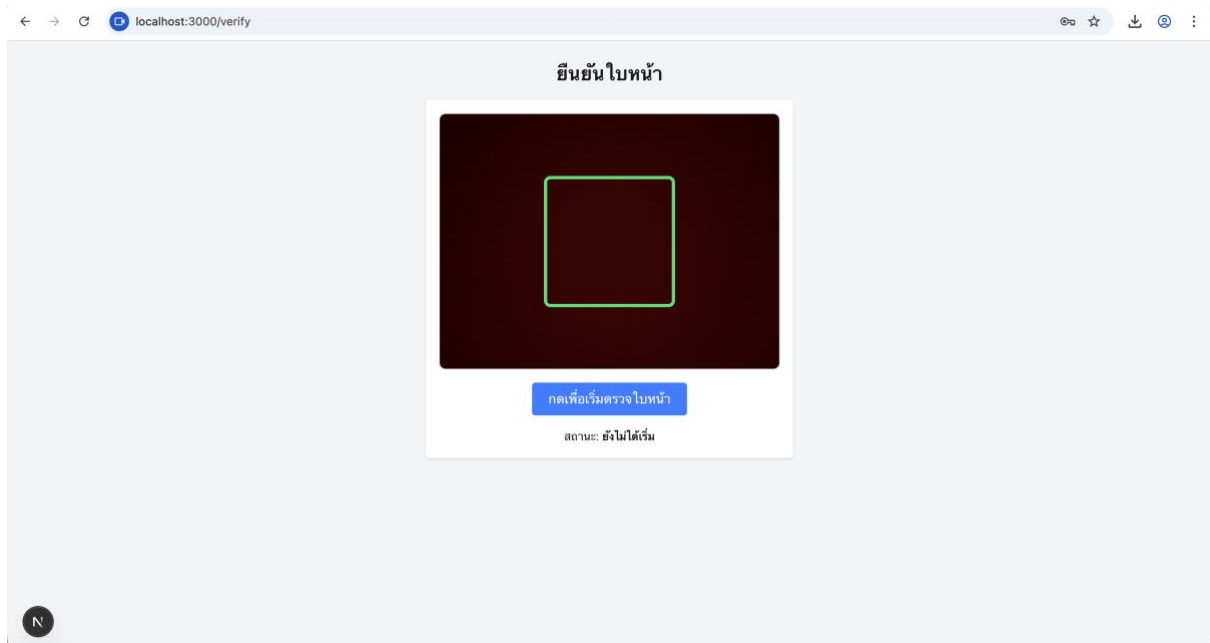
(รูปภาพประกอบข้อ12.1)

12.2 หน้าLogin



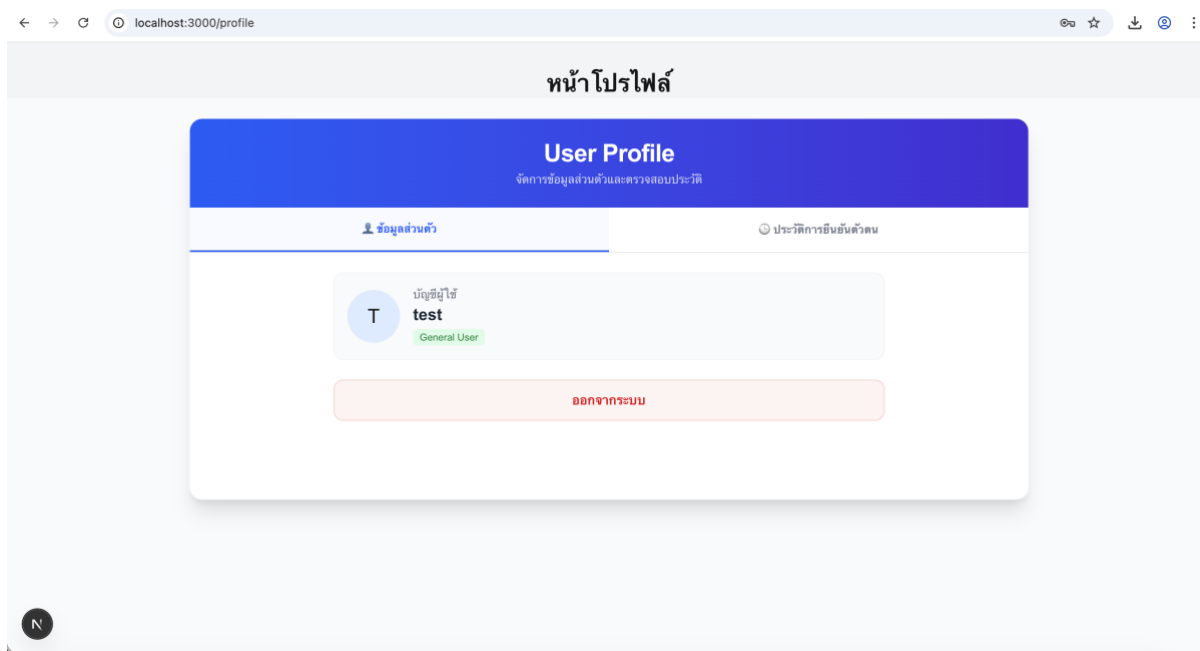
(รูปภาพประกอบข้อ12.2)

12.3 หน้าVerify



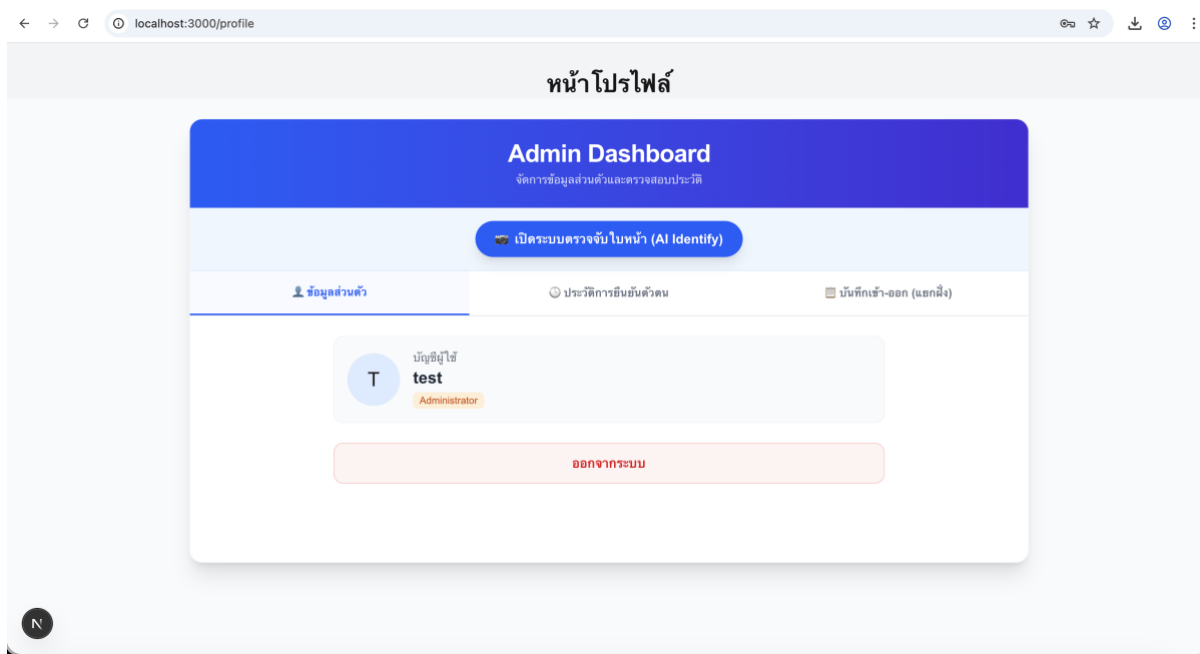
(รูปภาพประกอบข้อ12.3)

12.4 หน้าProfile



(รูปภาพประกอบข้อ12.4)

12.5 หน้าProfile สำหรับ Admin โดยจะมี Identify เพิ่มเข้ามา



(รูปภาพประกอบข้อ12.4)

ช่องทางการติดต่อ หากติดขัดหรือมีเรื่องสงสัย

Email : f.panuthep@gmail.com | Ig : fpnt.clv | Fb : panuthep chulavech